幕張 × 青木昆陽 WebAR & 画像生成アプリ

MVP 仕様書 v0.1

作成日: 2026-04-04

---

1. 目的

幕張地域を盛り上げるための、一般ユーザー向け Web アプリを構築する。

このアプリは次の 2 つを一体体験として提供する。

1. **WebAR 体験**
   - 現地または日常空間に青木昆陽を出現させる
   - フォトフレーム的 / XR 的に撮影を楽しめる
2. **画像生成体験**
   - 撮影した像、または照をもとに
   - 「幕張観光ポスター風」「江戸絵巻風」「さつまいも祭風」などの世界観に変換する

---

2. コンセプト定義

2.1 体験コンセプト

- 青木昆陽を“地域の案内人 / 応援キャラクター”として常に存在させる
- AR はその場で楽しい入口
- 画像生成は持ち帰れる成果物
- 幕張のスポットやイベントと結びつく回遊導線をつくる

2.2 コア原則

- **青木昆陽は毎回出る**
- **一般ユーザーが迷わない**
- **入力は簡単、裏側は賢く**
- **現地でも遠隔でも遊べる**
- **MVP では確実に動く範囲に絞る**

---

3. MVP のスコプ

3.1 含める機能

A. AR

- World 配置 AR
  - 床 / 地面 / テーブルに青木昆陽を立たせる
- Image Target AR
  - ポスター / 案内板 / 特定マーカーを読み取ると青木昆陽が出る
- Face / フォトフレーム AR
  - 自撮り向けフレーム
  - 青木昆陽の吹き出しや芋モチーフ演出

B. 画像生成

- AR で撮影した画像を素材として生成
- 手持ちをアップロドして生成
- 固定の青木昆陽アセットを毎回サーバ側で差し込み
- 情景 / 画風参照 / プリセット / 短い自由文で生成

C. 管理機能

- 青木昆陽アセット差し替え
- 画風プリセット管理
- 幕張スポットプリセット管理
- Image Target 管理
- 生成ジョブ監視

3.2 MVP では入れないもの

- VPS / 位置固定の街全体 AR
- 地図ベースの座標アンカー AR
- ユーザー独自キャラ学習
- 動生成
- 複雑なソーシャル機能
- EC / クーポン / 決済

---

4. 採用方針 (確定)

4.1 画像生成方針

- **本番既定モデル**: Gemini 2.5 Flash Image
- **検証用フラグ**: Gemini 3.1 Flash Image Preview
- **難ケース用の将来拡張**: Gemini 3 Pro Image Preview

4.2 AR 方針

- **8th Wall self-host 前提**
- WebAR は 8th Wall XR Engine + three.js ベース
- Next.js アプリと同一ドメイン配下で提供

4.3 プラットフォーム方針

- 最初は **Web アプリ / PWA**
- スマホブラウザ最適化優先
- ネイティブアプリ化は後段

---

5. 推奨技術スタック

5.1 フロント

- Next.js
- TypeScript
- React
- Tailwind CSS
- PWA 対応

5.2 AR

- 8th Wall XR Engine
- three.js
- 8th Wall Desktop / self-host build

5.3 バックエンド / データ

- Supabase Auth
- Supabase Postgres
- Supabase Storage
- Supabase Edge Functions

5.4 生成ワーカー

- Google Cloud Run
- Vertex AI API
- Gemini 2.5 Flash Image

5.5 運用補助

- Sentry
- PostHog or Plausible
- GitHub Actions

---

6. システムアーキテクチャ

```
[User Mobile Browser]
  ├─ Next.js Web App
  │  ├─ /          ホーム
  │  ├─ /ar        WebAR 導線
  │  ├─ /create     像生成導線
  │  ├─ /result/:id  結果表示
  │  └─ /admin      管理画面
  │
  └─ Camera / Upload / Share

[8th Wall XR Engine]
  └─ World / ImageTarget / Face AR

[Supabase]
  ├─ Postgres (セッション、ジョブ、プリセット、ログ)
  ├─ Storage  (raw, capture, generated, public-share)
  └─ Edge Functions (署名 URL、ジョブ作成、軽量 API)

[Cloud Run Worker]
  ├─ 画像前処理
  ├─ Prompt Builder
  ├─ Vertex AI 呼び出し
  ├─ 出力保存
  └─ ジョブ状態更新

[Vertex AI]
  └─ Gemini 2.5 Flash Image / 3.1 Flash Image Preview
```

---

7. モノレポ構成案

```
repo/
  apps/
    web/              # Next.js
    xr/               # 8th Wall project / build assets
    worker/           # Cloud Run generation worker
  packages/
    shared-types/
    prompt-builder/
    preset-engine/
    analytics/
  infra/
```

supabase/
gcp/

理由

- 8th Wall 側と Next.js 側のビルド責務を分離できる
- 生成ワーカーを独立させやすい
- プロンプト組み立てロジックを再利用しやすい

---

8. ユザ導線

導線 A: 現地 AR から生成

1. ホームを開く
2. 「AR で青木昆陽を出す」を押す
3. World 配置 or Target スキャン or Selfie を選ぶ
4. 画面内で撮影
5. 撮影画像をベースに画風を選ぶ
6. 画像生成
7. 保存 / 共有

導線 B: 写真から生成

1. ホームを開く
2. 「写真からつくる」を押す
3. 情景写真をアップロード
4. 画風 or スポットプリセットを選ぶ
5. ひとこと入力
6. 生成
7. 保存 / 共有

導線 C: スポット体験

1. 現地のポスター / 看板 / 案内板で QR を読む
2. 対象の Image Target AR を開く
3. 青木昆陽が出現
4. 撮影して生成へ進む

---

9. 画面一覧

9.1 一般ユーザー

- /
  - ヒーロー導線
  - 使い方
  - 3 つの入口（AR / ⊠⊠ / スポット）

- /ar
  - AR モード選択
- /ar/world
  - World 配置 AR
- /ar/target/[slug]
  - Image Target AR
- /ar/face
  - Selfie / フレーム AR
- /create
  - 画像生成入力
- /result/[jobId]
  - 生成結果表示
- /share/[slug]
  - 公開共有ページ
- /mypage（任意）
  - 過去生成一覧

9.2 管理画面

- /admin
  - ダッシュボード
- /admin/assets
  - 青木昆陽アセット管理
- /admin/styles
  - 画風プリセット管理
- /admin/spots
  - スポットプリセット管理
- /admin/targets
  - Image Target 管理
- /admin/jobs
  - 生成ジョブ監視
- /admin/reports
  - 通報確認

---

10. UX 原則

10.1 UI の基本方針

- プロンプト欄を主役にしない
- まず「どこで？」を選ばせる
- 次に「どんな雰囲気？」を選ばせる
- 写真があれば足す

- 最後に一言だけ自由入力

10.2 入力構成

- 固定（サーバ側）: 青木昆陽アセット
- ユーザー入力 1: 情景写真 or AR 撮影画像
- ユーザー入力 2: 画風参照（任意）
- ユーザー入力 3: スポットプリセット
- ユーザー入力 4: 画風プリセット
- ユーザー入力 5: 短い自由文

10.3 MVP で見せるアスペクト比

- 1:1
- 4:5
- 9:16

※ モデル側の対応比率はもっと広いが、UI は絞る

---

11. 画像生成仕様

11.1 基本ルール

- 青木昆陽は毎回必須
- ユーザーが青木昆陽画像を毎回アップロードする必要はない
- 青木昆陽アセットはサーバ側で自動注入

11.2 入力スロット定義

Slot A: Hero Asset（固定）

- 青木昆陽公式アセット
- ユーザー非表示
- バージョン管理あり

Slot B: Scene Image（任意 / 推）

- 現地写真
- AR キャプチャ
- 手持ち写真

Slot C: Style Reference（任意）

- 画風参考画像

Text Inputs

- spot_preset
- style_preset
- short_free_text

- aspect_ratio

11.3 生成優先順位

1. 青木昆陽アセット整合性
2. 構図 / 情景整合性
3. 画風整合性
4. 自由入力

11.4 モード

- ar_capture_generate
- photo_generate
- target_generate

11.5 Prompt Builder 仕様

- 入力 UI は日本語
- 内部プロンプトもまずは日本語ベース
- テンプレート + プリセット + 自由文を結合
- 禁止表現 / NG スタイルをサーバ側で除去
- user 入力は文字数制限

11.6 モデル選択ロジック

- 既定: gemini-2.5-flash-image
- 条件付きフラグ ON 時: gemini-3.1-flash-image-preview
- 管理者テスト時のみ: gemini-3-pro-image-preview

11.7 前理

- 元像は Supabase Storage へ保存
- 推論用に像を正規化
  - 長辺圧縮
  - JPEG / PNG 變
  - 7MB 以下に調整
- メタデータ（縦横、MIME、サイズ）を保存

11.8 出力

- MVP は 1 生成 1 枚を基本
- 将来はバリエーション複数枚に拡張
- 出力画像は Storage に保存
- 共有用派生画像も作成可能にする

---

12. AR 仕様

12.1 World 配置 AR

ゴール

- その場に青木昆陽を立たせる
- ユーザーは位置調整して撮影できる

UI

- カメラ起動
- 床検出待ち
- タップで配置
- ピンチで拡大縮小
- 回転 UI
- シャッターボタン

データ保存

- placement_mode = world
- pose_json
- scale
- rotation

12.2 Image Target AR

ゴール

- 現地ポスター / 説明板 / マーカーを起点に演出開始

UI

- スキャンガイド
- 認識成功で演出開始
- シャッターボタン
- 「このまま画像生成へ」CTA

Target 運用

- ターゲットごとに slug
- ターゲット画像管理
- 対応スポットプリセット紐付け

12.3 Face / フォトフレーム AR

ゴール

- 現地外でも遊べる軽い体験
- SNS 共有しやすい

UI

- フロントカメラ起動
- フレーム選択
- 吹き出し / 芋 / 幕張ロゴ演出
- シャッター

---

13. データモデル

13.1 sessions

- id
- user_id nullable
- anonymous_token
- created_at
- last_seen_at
- device_type
- locale

13.2 hero_assets

- id
- version
- name
- asset_type (image, sprite, 3d)
- storage_path
- thumbnail_path
- is_active
- metadata_json
- created_at

13.3 uploads

- id
- session_id
- kind (scene, style_ref, ar_capture, target_image, generated)
- bucket
- storage_path
- mime_type
- width
- height
- file_size
- visibility (private, public)

- created_at

13.4 spot_presets

- id
- slug
- name
- description
- prompt_template
- default_ar_mode
- is_active
- sort_order

13.5 style_presets

- id
- slug
- name
- description
- prompt_template
- is_active
- sort_order

13.6 ar_targets

- id
- slug
- name
- target_upload_id
- spot_preset_id
- is_active
- metadata_json

13.7 ar_captures

- id
- session_id
- mode (world, target, face)
- capture_upload_id
- target_id nullable
- pose_json
- created_at

13.8 generation_jobs

- id

- session_id
- source_capture_id nullable
- scene_upload_id nullable
- style_upload_id nullable
- hero_asset_version
- spot_preset_id nullable
- style_preset_id nullable
- free_text
- aspect_ratio
- provider
- model_id
- status
- error_code nullable
- error_message nullable
- request_json
- timing_json
- created_at
- started_at
- completed_at

13.9 generation_outputs

- id
- job_id
- upload_id
- variant_no
- is_selected
- has_c2pa
- created_at

13.10 share_pages

- id
- output_id
- slug
- is_public
- expires_at nullable
- created_at

13.11 reports

- id
- output_id

- reason
- status
- created_at

13.12 analytics_events

- id
- session_id
- event_name
- event_json
- created_at

---

14. Storage 設計

バケット

- raw-private
  - 元アップロード
- capture-private
  - AR キャプチャ
- generated-private
  - 生成結果原本
- public-share
  - 共有用画像
- admin-assets
  - 木昆陽素材 / ターゲット素材

原則

- 個人は private
- 共有操作したものだけ public-share に派生
- 保存期間ポリシーを管理画面で変更可能にする

---

15. ジョブ状態遷移

queued
-> preprocessing
-> prompt_built
-> generating
-> storing
-> completed
-> failed
-> blocked

blocked 条件

- 安全フィルタ
- 禁止ワード
- 管理者ルール違反

---

16. API 仕様（MVP）

16.1 公開 API

GET /api/bootstrap

返すもの - active hero asset version - spot presets - style presets - feature flags

POST /api/uploads/sign

用途 - 画像アップロード用署名 URL

POST /api/ar/captures

用途 - AR キャプチャ記録

POST /api/generation/jobs

用途 - 生成ジョブ作成

入力例

```
{
  "source_type": "ar_capture",
  "capture_id": "cap_123",
  "scene_upload_id": null,
  "style_upload_id": "upl_456",
  "spot_preset_id": "spot_makuhari_beach",
  "style_preset_id": "style_ukiyoe",
  "free_text": "夕暮れのポスター風で",
  "aspect_ratio": "4:5"
}
```

返却例

```
{
  "job_id": "job_123",
  "status": "queued"
}
```

GET /api/generation/jobs/:id

返すもの - status - preview state - output image URLs (completed 時)

POST /api/share

用途 - 共有ページ生成

16.2 管理 API

- POST /api/admin/hero-assets
- POST /api/admin/style-presets
- POST /api/admin/spot-presets
- POST /api/admin/ar-targets
- GET /api/admin/jobs
- POST /api/admin/jobs/:id/retry

16.3 内部 API

- POST /internal/process-job
- POST /internal/rebuild-share-image
- POST /internal/reconcile-failed-jobs

---

17. 生成ワーカー仕様

入力

- job_id

処理

1. DB からジョブ取得
2. hero asset 決定
3. scene/style 画像読込
4. 推論用画像正規化
5. prompt 組み立て
6. Vertex AI 呼び出し
7. 出力保存
8. C2PA 有無記録
9. DB 更新

エラー時

- リトライ回数を制限
- Safety blocked は retry しない
- 一時障害は指数バックオフ

---

18. 管理画面仕様

18.1 Hero Asset 管理

- 有効バージョン切替
- サムネイル確認
- 利用履確認

18.2 Spot Preset 管理

- スポット名
- 説明文
- プロンプトテンプレート
- 推奨 AR モード
- 表示順

18.3 Style Preset 管理

- 画風名
- サンプル画像
- プロンプトテンプレート
- 表示順

18.4 AR Target 管理

- ターゲット画像登録
- slug 設定
- 紐付けスポット
- 有効 / 無効

18.5 Job 監視

- ステータス一覧
- 失敗理由
- 再実行
- モデル別成功率

---

19. 安全性運用ポリシ

19.1 入力制御

- 自由文は短文のみ
- 禁止語辞書
- 公序良俗 NG
- 個人攻撃 / 差別 / 露骨表現 NG

19.2 出力制御

- モデル安全フィルタ利用

- blocked 出力は保存しない
- 通報導線を置く

19.3 透明性

- AI 生成ラベルを表示
- 共有ページにも AI 生成表記
- C2PA が取得できる場合は保持

19.4 プライバシー

- 個人写真は private bucket 保管
- 公開共有はユーザー明示操作でのみ
- 保持期間を設計で明記

---

20. ログ / 分析イベント

記録するイベント例 - app_opened - ar_opened - ar_mode_selected - placement_completed - target_detected - capture_saved - generate_started - generate_completed - generate_failed - share_created - share_opened - report_submitted

---

21. 受け入れ件 (MVP 完了件)

AR

- スマホブラウザで World 配置 AR が動く
- Image Target AR が 1 種類以上動く
- Selfie 用フレーム AR が 1 種類以上動く

生成

- AR 撮影画像から生成できる
- 写真アップロードから生成できる
- 青木昆陽が毎回入る
- 生成結果を保存 / 共有できる

管理

- 青木昆陽アセット差し替えができる
- スタイル / スポット / ターゲット管理ができる
- ジョブ失敗を追える

---

22. 実装上の重要判断

22.1 2.5 系の 3 画像制限を前提に UI を作る

- 固定枠: 木昆陽

- ユーザー枠 1: 情景 or AR 撮影
- ユーザー枠 2: 画風参照（任意）

22.2 2.5 の寿命を見越して Provider 抽象化する

- ImageProvider.generate() インターフェースを定義
- 2.5 -> 3.1 切り替えを容易にする

22.3 8th Wall は別アプリ責務で持つ

- apps/xr として分離
- apps/web とは URLセッションを共有

22.4 生画像と共有画像を分ける

- 元画像は private
- 共有用のみ public 化

---

23. 将来拡張

- スポットごとの限定演出
- イベント連動キャンペーン
- スタンプラリー
- 多言語化
- 観光スポット解説音声
- Location / VPS 系 AR
- 動画成
- 印刷 / ポストカード出力

---

24. 次にやるべき具体タスク

1. 画面ワイヤーフレーム作成
2. 青木昆陽アセット仕様定義
3. Spot Preset 初期案作成
4. Style Preset 初期案作成
5. DB DDL 作成
6. API I/O を OpenAPI に落とす
7. apps/web / apps/xr / worker の雛形作成
8. Vertex AI 呼び出し PoC
9. 8th Wall World 配置 PoC
10. AR キャプチャー生成の縦スライス実装